PROJECT EXPLAINED IN SIMPLE TERMS

A 99.99%-Available Multi-Tenant Data Platform on Oracle Cloud (OCI)

> **The resume line this document explains**
>
> "Achieved 99.99% availability for a multi-tenant platform spanning 12K+ nodes, as measured by SLO dashboards, by leading design of Java and Go data-plane microservices on OCI and defining scalability, elasticity, and durability requirements."

What's inside: a plain-English summary, a glossary, a full architecture diagram with a walkthrough, a detailed card for each of the 23 components, eight end-to-end workflows, the requirements that were defined, the 99.99% maths, and a short interview script.

Note: *the resume line gives the results, not the internal design. The components below form a realistic reference architecture consistent with that line. Rename them to match your real system where needed.*

Contents

1. The Big Picture (Simple Summary) Plain-English overview, analogy, headline numbers

2. Key Terms in Plain English Glossary of every technical word used

3. Architecture Diagram Full diagram, how to read it, layer-by-layer walkthrough, design ideas

4. Every Component, Explained 23 component cards + why Java and Go

5. End-to-End Workflows Write, background, read, onboarding, scaling, failures, deployment, measurement

6. The Requirements I Defined Availability, scalability, elasticity, durability, latency, isolation, recovery

7. The 99.99% in Numbers Downtime maths, error budget example, N+1 capacity

8. How to Explain This in an Interview 30-second pitch, STAR summary, follow-up answers

1. The Big Picture (Simple Summary)

> **In one paragraph**
>
> We built and ran a large shared cloud platform where many customers (**tenants**) store and fetch their data. It runs on more than **12,000 computers (nodes)** in Oracle Cloud. I led the design of the services that handle every customer request (the **data plane**), written in **Java and Go**, and I wrote down the rules for how the system must grow (**scalability**), shrink and expand automatically (**elasticity**) and never lose data (**durability**). Because of this design, the platform was up and working **99.99% of the time**, which means it was allowed to be down for only about **4 minutes per month**, and we proved it with **SLO dashboards** that measure every request.

1.1 The everyday analogy

Think of the platform as a **huge apartment building with a 24-hour concierge service**:

- **Tenants** are the families living in the building. They share the building but must never see each other's belongings.
- The **front door, security guard and reception** (DNS, WAF, Load Balancer, API Gateway) let only the right people in and send them to the right place.
- The **fair-use rules** (Rate Limiter) stop one family from using all the hot water.
- The **storerooms and vaults** (cache, database, queue, object storage) keep everyone's belongings, with copies in three separate buildings so nothing is lost in a fire.
- The **management office** (control plane) makes the rules, but the building keeps working even if the office is closed for a day.
- The **CCTV and alarm system** (observability) watches everything and calls the repair team before residents notice a problem.

1.2 The headline numbers

Item	Value	What it means in simple terms
Availability	99.99% ("four nines")	Out of every 10,000 requests, at most 1 may fail. About 4.3 minutes of total downtime allowed per month.
Scale	12,000+ nodes	More than twelve thousand machines running the platform's services.
Tenancy	Multi-tenant	Many customers share the same platform, safely isolated from each other.
Languages	Java and Go	Java for rich business logic and transactions; Go for fast, lightweight, high-throughput services.

Item	Value	What it means in simple terms
Cloud	Oracle Cloud Infrastructure (OCI)	The cloud provider hosting the machines, networks, databases and storage.
Proof	SLO dashboards	Availability is measured from real request data, not guessed.

1.3 What "leading the design" covered

- Designing the request-handling (data-plane) microservices: gateway, rate limiter, router, read, write and worker services.
- Defining measurable **scalability, elasticity and durability requirements** that every team had to meet.
- Choosing reliability patterns: cells, quorum writes, idempotency, retries with backoff, circuit breakers, static stability, safe deployments.
- Defining SLIs/SLOs and dashboards so the 99.99% could be proven objectively.

2. Key Terms in Plain English

Read this once; every later section uses these words.

Term	Simple meaning
Multi-tenant	One platform shared by many customers. Each customer (tenant) only sees its own data.
Node	One computer (virtual or physical) in the cloud.
Microservice	A small program that does one job well and talks to other services over the network.
Data plane	The part of the system that handles real customer requests every second (the "engine").
Control plane	The part that manages settings, tenants, keys and deployments (the "steering wheel"). It is not in the path of each request.
Availability	The percentage of time (or requests) the service works correctly.
SLI	Service Level Indicator: the thing you measure, e.g., % of successful requests.
SLO	Service Level Objective: the target for the SLI, e.g., 99.99% over 30 days.
SLA	Service Level Agreement: a contract promise to customers (usually looser than the internal SLO).
Error budget	The allowed amount of failure: 100% − SLO. For 99.99%, that is 0.01% of requests.
Scalability	Ability to handle more load by adding more machines (growing).
Elasticity	Ability to grow AND shrink automatically and quickly as load changes.
Durability	Once data is saved, it is never lost.
Region / AD / FD	Region = a geographic area. Availability Domain (AD) = an independent data centre inside a region. Fault Domain (FD) = an independent hardware group inside an AD.
OKE / Kubernetes / Pod	Kubernetes runs programs in containers; a pod is one running copy. OKE is Oracle's managed Kubernetes.
Cell	A self-contained slice of the platform (services + data) serving a subset of tenants, so failures stay contained.
Quorum	A majority of copies (e.g., 2 of 3) must confirm before a write is considered done.

Term	Simple meaning
Idempotent	Doing the same operation twice has the same effect as doing it once. Makes retries safe.
Latency / p99	How long a request takes. p99 = 99% of requests are faster than this value.
MTTR	Mean Time To Recover: how quickly service is restored after a problem.

3. Architecture Diagram

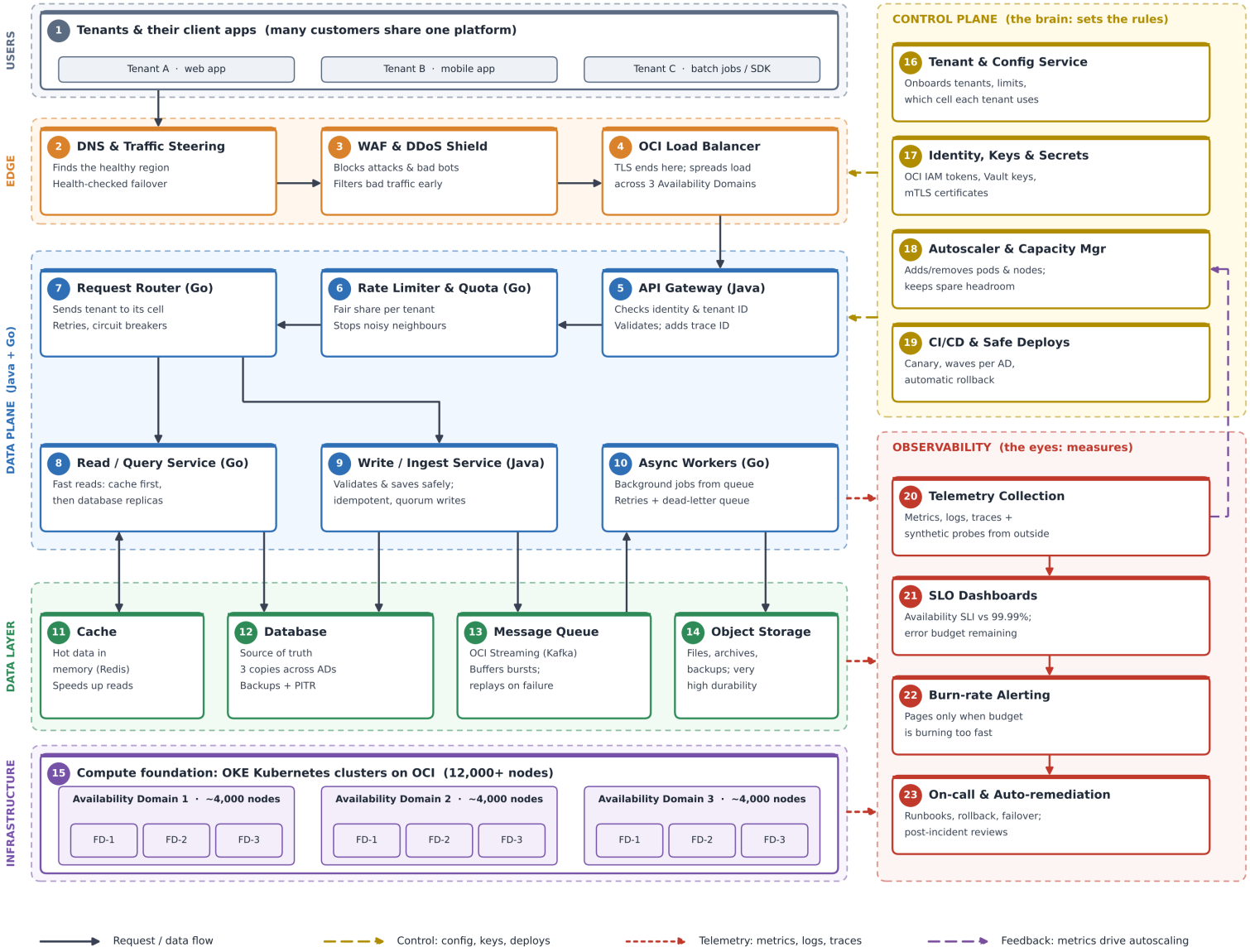


Figure 1. End-to-end architecture. Numbers (1–23) match the component cards in Section 4.

3.1 How to read the diagram

- **Top to bottom = the path of a request**: users → edge → data plane → data layer → infrastructure.
- **Right side = the helpers**: the control plane (yellow) that sets the rules, and observability (red) that measures everything.
- **Colours = layers**: grey users, orange edge, blue data plane, green data layer, purple infrastructure, yellow control, red observability.
- **Numbered circles** identify each component; Section 4 explains each one in the same order.
- **Arrow types** (see legend at the bottom of the diagram): solid black = requests/data; yellow dashed = configuration, keys and deployments; red dotted = telemetry; purple dashed = metrics feeding the autoscaler.

3.2 Walking through the diagram, layer by layer

Diagram area	What you see and what it means
Users (#1)	Tenants A, B and C use web apps, mobile apps and batch jobs. They all share one platform. One arrow leaves this layer: every request goes to DNS first.
Edge (#2 → #3 → #4)	Left to right: DNS finds a healthy entry point → WAF blocks attacks → Load Balancer decrypts HTTPS and spreads requests across 3 ADs. The arrow from #4 goes down into the data plane.
Data plane, top row (#5 → #6 → #7)	Read right to left: the API Gateway (Java) checks identity and tenant → the Rate Limiter (Go) enforces fair use → the Router (Go) finds the tenant's cell.
Data plane, bottom row (#8, #9, #10)	The Router sends reads straight down to the Read Service (#8) and writes, via the elbow arrow, to the Write Service (#9). Async Workers (#10) are fed by the queue, not by the Router.
Data layer (#11–#14)	Vertical arrows show who talks to whom: Read ↔ Cache (two-way: read and refill) and Read → Database; Write → Database and Write → Queue; Queue → Workers (arrow points up: workers pull events); Workers → Object Storage.
Infrastructure (#15)	Everything above runs on OKE Kubernetes clusters: 12,000+ nodes split across 3 Availability Domains (~4,000 each), each with 3 Fault Domains (FD-1, FD-2, FD-3).
Control plane (#16–#19)	Tenant config, identity/keys, autoscaling and deployments. The yellow dashed arrows show it pushing configuration, keys and releases into the edge and data plane. It is not on the request path.

Diagram area	What you see and what it means
Observability (#20 → #23)	Red dotted arrows carry telemetry from the data plane, data layer and infrastructure into collection (#20) → SLO dashboards (#21) → burn-rate alerting (#22) → on-call and auto-remediation (#23). The purple dashed loop shows metrics from #20 driving the autoscaler (#18).

3.3 The design ideas the diagram encodes

Design idea	Where it appears in the diagram and why it matters
Redundancy everywhere	Every box runs as many copies across 3 ADs and 3 FDs (#15). One failure never takes a component fully offline.
Data plane / control plane split	Yellow panel is separate from the blue data plane. Requests never wait on the control plane, so its outages don't cause customer outages.
Cells (bulkheads)	The Router (#7) sends each tenant to its own cell. A failure in one cell affects only a small group of tenants.
Fail-fast at the edge	WAF (#3), Gateway (#5) and Rate Limiter (#6) reject bad or excessive traffic before it uses expensive resources.
Sync vs async	Only the essential work (save data durably) happens while the tenant waits. Heavy work goes through the Queue (#13) to Workers (#10).
Durable by design	Quorum writes to the Database (#12), replicated Queue (#13), and highly durable Object Storage (#14) with backups.
Measure, then act	Telemetry → SLOs → alerts → remediation (#20–#23), plus the feedback loop to autoscaling (#18).

4. Every Component, Explained

Each card follows the same pattern: what it is in one line, an everyday analogy, exactly what it does, how it helps reach 99.99%, and the technology used. Card colours match the diagram layers.

4.1 Users

#1 Tenants & their client apps	
In one line	The customers (companies/teams) who use the platform, and the apps they send requests from.
Simple analogy	Like families living in one big apartment building: everyone shares the same building, water and lifts, but each family has its own locked flat.
What it does, step by step	• A **tenant** is one customer account (for example, Company A). Each tenant gets a unique tenant ID. • Tenants reach the platform from web apps, mobile apps, scripts, SDKs or scheduled batch jobs. • Every request carries an **access token** (proof of who they are) and is sent over **HTTPS** (encrypted). • Well-behaved clients **retry with backoff and jitter** (wait a little longer each time, with a bit of randomness) and send an **idempotency key**, so a retried request is never processed twice.
How it protects 99.99%	• Because many tenants share one platform, one tenant's problem must never become everyone's problem; that is why isolation features appear in later components. • Availability (99.99%) is measured from the tenant's point of view: did their request succeed?
Built with	Any client: browsers, mobile apps, REST/gRPC SDKs, CLI tools.

4.2 Edge layer

#2 DNS & Traffic Steering	
In one line	The internet's phone book: turns the platform's name (e.g., api.example.com) into the address of a healthy entry point.
Simple analogy	A GPS that only sends you down roads that are currently open.
What it does, step by step	• When a client looks up the API name, DNS answers with the IP address of the nearest healthy region/load balancer. • **Health checks** constantly ping each entry point. If one stops answering, DNS stops handing out its address (**failover**). • A **short TTL** (time-to-live, e.g., 30–60 seconds) means clients refresh the answer quickly, so a failover takes effect fast. • Can steer by geography (send users to the closest region) or by weight (send 5% of traffic to a new region).
How it protects 99.99%	• Removes a whole region or entry point from rotation automatically during a big outage. • Uses a highly redundant managed service, so DNS itself is not a single point of failure.
Built with	OCI DNS with Traffic Management Steering Policies and OCI Health Checks.

#3 WAF & DDoS Shield	
In one line	A security guard at the front door that blocks attacks and junk traffic before it reaches the services.
Simple analogy	Airport security screening: dangerous items are stopped before anyone reaches the gate.
What it does, step by step	• **WAF** (Web Application Firewall) inspects requests and blocks known attack patterns such as SQL injection or cross-site scripting. • Blocks abusive bots and suspicious IP addresses; applies coarse rate limits per IP. • **DDoS protection** absorbs floods of fake traffic meant to overwhelm the platform. • Only clean traffic is passed on to the load balancer.
How it protects 99.99%	• Attack traffic can eat capacity and cause outages. Filtering it at the edge protects the 99.99% target. • Inner services spend their capacity only on real tenant work.
Built with	OCI Web Application Firewall (WAF) and OCI's built-in DDoS protection.

#4 OCI Load Balancer	
In one line	Spreads incoming requests evenly over many copies of the API Gateway, and only sends traffic to healthy copies.
Simple analogy	A supermarket manager directing shoppers to the shortest open checkout lane.
What it does, step by step	• **Terminates TLS**: decrypts HTTPS once here, so services behind it get plain, fast traffic inside a private, secured network (or re-encrypts with mTLS). • Distributes requests across API Gateway instances running in **all three Availability Domains (ADs)**. • Runs health checks every few seconds; an unhealthy instance is removed from rotation automatically. • **Connection draining**: during deployments, an instance being removed finishes its in-flight requests first, so no request is cut off.
How it protects 99.99%	• No single server failure is visible to tenants; traffic simply goes to the remaining healthy ones. • The load balancer itself is a managed, highly available pair, spread across fault domains.
Built with	OCI Flexible Load Balancer (Layer 7) or Network Load Balancer (Layer 4).

4.3 Data plane microservices (Java + Go)

#5 API Gateway / Front Door (Java)	
In one line	The first service of the platform's own code. It checks who is calling, which tenant they belong to, and whether the request is valid.
Simple analogy	A hotel reception desk: checks your ID and booking before giving you a room key.
What it does, step by step	• **Authenticates** the caller by verifying the token's signature with public keys cached from Identity (#17). No network call per request, so it is fast and still works if Identity is briefly down. • Extracts the **tenant ID** and attaches it to the request so every later service knows whose request it is. • **Validates** the request (correct format, size limits, required fields) and rejects bad ones early with a clear 4xx error. • Adds a **request/trace ID** so the request can be followed through every service in logs and traces. • Sets a **deadline** (total time budget). Every downstream call shares this budget, so requests never hang forever.
How it protects 99.99%	• Stateless: any copy can handle any request, so copies can be added or removed freely. • Runs many copies across 3 ADs, behind the load balancer.
Built with	Java (e.g., Spring Boot / Netty-based async HTTP), JWT verification, OpenTelemetry.

#6 Rate Limiter & Quota Service (Go)	
In one line	Makes sure each tenant only uses its fair share, so one busy tenant cannot slow down everyone else.
Simple analogy	A buffet rule: everyone can take food, but no one may empty the whole tray at once.
What it does, step by step	• Uses a **token bucket** per tenant: each tenant gets N tokens per second; every request spends one; empty bucket means wait. • Over-limit requests get **HTTP 429 Too Many Requests** with a **Retry-After** header telling the client when to try again. • Counters are kept locally in memory for speed and synchronised through the cache (#11) for a global view. • **Load shedding**: if the whole platform is overloaded, low-priority work (e.g., bulk exports) is rejected first to protect critical traffic. • Limits per tenant come from their plan, stored in the Tenant & Config Service (#16).
How it protects 99.99%	• Prevents the **noisy-neighbour** problem, the main risk of multi-tenant systems. • Fails open safely: if the shared counter store is unreachable, it falls back to local limits rather than blocking all traffic.
Built with	Go (lightweight, very fast, low memory), Redis-compatible cache for shared counters.

#7 Tenant-aware Request Router (Go)	
In one line	Decides exactly which group of servers (cell) should handle this tenant's request, and calls it safely.
Simple analogy	A postal sorting office: each letter goes to the right local post office based on its postcode.
What it does, step by step	• Looks up the tenant in a **routing map** (tenant → cell/shard), cached locally from the Config Service (#16). • A **cell** is an independent, complete copy of the services and data serving a subset of tenants. A problem in one cell affects only its tenants (**blast-radius control**). • Sends reads to the Read Service (#8) and writes to the Write Service (#9). • **Retries** failed calls a limited number of times with exponential backoff and jitter, only for safe (idempotent) operations. • **Circuit breaker**: if a downstream copy keeps failing, it stops calling it for a while and lets it recover. • **Hedged reads**: if a read is slow, it may send a second copy of it to another replica and use whichever answers first.
How it protects 99.99%	• Keeps working on the **last-known-good** routing map even if the control plane is down (**static stability**). • Avoids sending traffic to unhealthy instances and stops retry storms.
Built with	Go, gRPC, consistent hashing, client-side load balancing.

#8 Read / Query Service (Go)	
In one line	Answers "get me my data" requests as fast as possible.
Simple analogy	A librarian who keeps the most popular books on the front desk instead of fetching them from the basement every time.
What it does, step by step	• Checks the **cache** (#11) first (cache-aside pattern). On a hit, returns immediately. • On a miss, reads from a **database replica** (#12), returns the result and puts it in the cache for next time. • Every query is filtered by **tenant ID**, so a tenant can only ever see its own data. • If the cache is unavailable, it goes straight to the database (slower, but still correct).

How it protects 99.99%	• Reads are spread across replicas in different ADs; losing one replica does not stop reads. • Stateless and horizontally scalable: more copies are added automatically under load.
Built with	Go, Redis client, database driver with connection pooling.

#9 Write / Ingest Service (Java)	
In one line	Accepts new or changed data and saves it so it is never lost.
Simple analogy	A bank teller who doesn't hand you a receipt until your deposit is recorded in the ledger in more than one place.
What it does, step by step	• Validates the business rules of the data (e.g., required fields, allowed values). • Checks the **idempotency key**: if this exact request was already processed (e.g., a client retry), it returns the original result instead of saving twice. • Writes to the database (#12) as a transaction. The write counts as done only when a **quorum** (e.g., 2 of 3 copies in different ADs) confirms it. • Uses the **transactional outbox** pattern: the data and a matching "event" row are saved in the same transaction; a relay then publishes the event to the queue (#13). Data and event never disagree. • Only after the durable commit does it reply **201 Created / 200 OK** to the tenant.
How it protects 99.99%	• **Durability**: an acknowledged write survives the loss of any one server or even an entire AD. • Idempotency makes retries safe, which is what lets the rest of the system retry aggressively.
Built with	Java (strong transaction libraries, mature DB drivers), JDBC / NoSQL SDK, outbox relay.

#10 Async Workers (Go)	
In one line	Background workers that do slower follow-up jobs without making the tenant wait.
Simple analogy	A restaurant kitchen: the waiter takes your order instantly; cooks prepare it in the back.
What it does, step by step	• Read events from the message queue (#13): for example indexing, aggregation, notifications, exports, archiving files to Object Storage (#14). • Each job is **idempotent**: running it twice gives the same result, so redelivery is harmless. • On failure, retry with backoff. After several failures, the event goes to a **dead-letter queue (DLQ)** for investigation, so one bad message cannot block the rest. • **Acknowledge** (commit the offset) only after the job is fully done, so a crash mid-job means the job is simply redone. • Scale out automatically based on **queue lag** (how many events are waiting).
How it protects 99.99%	• Moves heavy work off the request path, keeping the tenant-facing path fast and simple. • A burst of events just queues up; nothing is dropped.
Built with	Go (goroutines handle thousands of concurrent jobs cheaply), Kafka-compatible consumer.

4.4 Data layer

#11 Cache	
In one line	Very fast in-memory storage of frequently used data.
Simple analogy	Your browser remembering a page so it opens instantly the second time.
What it does, step by step	• Stores hot data (recent reads, tenant settings, rate-limit counters) in memory. • Each entry has a **TTL** (expiry time) so stale data disappears automatically. • Keys include the tenant ID (e.g., tenantA:item:42) to keep tenants separated. • Runs as a replicated cluster (primary + replicas across ADs).
How it protects 99.99%	• Removes most read load from the database, so the database is never overwhelmed. • It is **not** the source of truth: if the cache is lost, data is reloaded from the database.

Built with	OCI Cache (Redis-compatible).

#12 Database (source of truth)

In one line	Where the real, permanent data lives.
Simple analogy	The bank's main ledger, with certified copies kept in three different vaults.
What it does, step by step	• Data is split into **shards/partitions** (by tenant or key range) so it can grow beyond one machine. • Every shard is **replicated** to 3 copies, one per Availability Domain. Writes are confirmed by a quorum. • Read replicas serve read traffic; the primary handles writes. If the primary fails, a replica is **promoted automatically** in seconds. • **Automated backups** plus **point-in-time recovery (PITR)** let data be restored to any moment (e.g., just before an accidental delete). • Backups are copied to another region for disaster recovery.
How it protects 99.99%	• Survives the loss of a disk, a server or a whole AD without losing acknowledged data. • Sharding stops one large tenant from filling a single database.
Built with	e.g., OCI Autonomous Database / MySQL HeatWave HA / OCI NoSQL (depending on data model).

#13 Message Queue / Streaming

In one line	A durable to-do list of events between the Write Service and the Workers.
Simple analogy	An order ticket rail in a kitchen: tickets wait in line until a cook picks them up.
What it does, step by step	• Stores events in **partitions** (keyed by tenant so each tenant's events stay in order). • Each message is replicated across fault domains so it is not lost if a broker fails. • Keeps messages for a **retention period** (e.g., 7 days), so events can be **replayed** after a bug fix. • **Decouples** producers from consumers: the Write Service never waits on the Workers.
How it protects 99.99%	• Absorbs traffic spikes: writes stay fast even if workers are temporarily slow. • If workers crash, messages wait safely until they recover.

Built with	OCI Streaming (Kafka-compatible API).

#14 Object Storage

In one line	Cheap, extremely durable storage for large files, exports, archives and backups.
Simple analogy	A secure warehouse for boxes you rarely open but must never lose.
What it does, step by step	• Stores large objects (files, logs, exports, database backups) organised in buckets per tenant/purpose. • Data is automatically stored redundantly across multiple fault domains/ADs and continuously checked for corruption (checksums, self-healing). • **Lifecycle rules** move old data to cheaper Archive tier automatically. • **Cross-region replication** keeps a copy in another region for disasters.
How it protects 99.99%	• Designed for very high durability (OCI states 99.999999999%, "eleven nines"). • Holds the backups that make database recovery possible.
Built with	OCI Object Storage (Standard, Infrequent Access and Archive tiers).

4.5 Infrastructure

#15 Compute foundation: OKE clusters (12,000+ nodes)	
In one line	The physical/virtual machines and the Kubernetes system that runs all the services.
Simple analogy	The land, buildings and electricity of a city, organised into three separate districts with independent power.
What it does, step by step	• A **node** is one virtual or bare-metal machine. The platform runs on **12,000+ nodes**. • **OKE** (Oracle Kubernetes Engine) runs each service as many small containers called **pods**, restarting them automatically if they crash. • An **Availability Domain (AD)** is an independent data centre in a region (own power, cooling, network). Workloads are spread across 3 ADs. • Each AD has 3 **Fault Domains (FDs)**: groups of hardware that do not share a single point of failure. Pods are spread across FDs too. • **Topology spread constraints** and **anti-affinity** make Kubernetes place copies of the same service on different nodes/FDs/ADs. • **PodDisruptionBudgets** stop maintenance from taking down too many copies of a service at once. • The fleet is split into many clusters/**cells**, so one cluster problem never affects the whole platform.
How it protects 99.99%	• Losing a node, an FD or even a full AD leaves enough copies running elsewhere. • Capacity is planned with **N+1 headroom**: with 3 ADs, each runs at most ~66% busy, so 2 ADs can carry 100% of traffic if one fails.
Built with	OCI Compute, OKE (Kubernetes), node pools, cluster autoscaler.

4.6 Control plane

#16 Tenant & Config Service	
In one line	The admin system that knows every tenant: their plan, limits and which cell they live in.
Simple analogy	The building's management office that keeps the list of residents and the rules.
What it does, step by step	• **Onboards** new tenants: creates the tenant ID, assigns a cell/shard, sets quotas. • Stores **feature flags** and configuration (timeouts, limits, routing maps). • **Pushes** configuration to the data plane, which caches it locally. • Validates config changes and rolls them out gradually, just like code.

How it protects 99.99%	• **Data plane does not depend on it at request time.** If the control plane is down, existing tenants keep working on the last-known-good config; only changes (like onboarding) pause. • This separation is one of the biggest reasons the data plane can reach 99.99%.
Built with	Java/Go service, versioned config store, push/watch mechanism.

#17 Identity, Keys & Secrets

In one line	Manages who is allowed to do what, and keeps passwords, keys and certificates safe.
Simple analogy	The key-card office: issues badges and changes the locks regularly.
What it does, step by step	• **OCI IAM** issues and signs access tokens for tenants and services. • **OCI Vault** stores encryption keys and secrets (database passwords, API keys) and **rotates** them automatically. • Issues **mTLS certificates** so services prove their identity to each other and all internal traffic is encrypted. • Data is encrypted at rest with keys from Vault (optionally per-tenant keys).
How it protects 99.99%	• Services cache public keys and certificates, so a short Identity outage does not break requests. • Rotating credentials automatically avoids outages caused by expired certificates (a classic cause of downtime).
Built with	OCI IAM, OCI Vault (KMS), certificate service.

#18 Autoscaler & Capacity Manager	
In one line	Automatically adds or removes computing power as traffic goes up and down (elasticity).
Simple analogy	A shop manager who opens more checkout lanes when the queue grows and closes them when it is quiet.
What it does, step by step	• **Horizontal Pod Autoscaler (HPA)** adds/removes pods based on CPU, memory, requests per second or queue lag. • **Cluster Autoscaler** adds/removes nodes when pods cannot fit or nodes sit idle. • Keeps **warm headroom** (spare capacity) so a sudden spike is absorbed before new nodes are ready. • Uses traffic history for **predictive scaling** before known peaks (e.g., Monday mornings, month-end). • Enforces **N+1 AD capacity**: always enough to lose one AD.
How it protects 99.99%	• Prevents overload outages during spikes; reduces cost during quiet periods. • Has min/max limits and scale-down cool-downs to avoid flapping (scaling up and down rapidly).
Built with	Kubernetes HPA, Cluster Autoscaler, custom metrics (Prometheus adapter).

#19 CI/CD & Safe Deployments	
In one line	The pipeline that builds, tests and releases new code without breaking things.
Simple analogy	A new recipe is first served to one table, then the whole restaurant, only if the first table loved it.
What it does, step by step	• Builds the code, runs unit, integration and load tests automatically. • Deploys to a **canary** first (one cell / small % of traffic) and watches error rate and latency during a **bake time**. • If health metrics get worse, it **rolls back automatically**. • Releases in **waves**: one AD at a time, then one region at a time, never everywhere at once. • Uses **feature flags** so new behaviour can be switched off instantly without redeploying.
How it protects 99.99%	• Most outages come from changes; safe deployment is the biggest single lever for 99.99%. • Limits the blast radius of a bad release to a small slice of traffic for a few minutes.

Built with	CI/CD pipelines (e.g., OCI DevOps / Jenkins / GitHub Actions), Helm/Argo-style rollouts.

4.7 Observability & operations

#20 Telemetry Collection	
In one line	Collects the numbers, logs and traces that show how the system is behaving.
Simple analogy	The dashboard and sensors in a car: speed, fuel, engine temperature.
What it does, step by step	• **Metrics**: counts and timings (requests, errors, latency percentiles, CPU) per service, per cell, per tenant. • **Logs**: detailed text records of events, tagged with request ID and tenant ID. • **Traces**: the full journey of one request across all services, showing where time was spent. • **Synthetic probes**: robot clients outside the platform that call the API every minute, measuring availability exactly as a tenant would see it.
How it protects 99.99%	• You cannot improve what you cannot see; this is the raw data behind the 99.99% number. • Collection is asynchronous and buffered, so telemetry problems never slow down requests.
Built with	OpenTelemetry, Prometheus / OCI Monitoring, OCI Logging, OCI APM.

#21 SLO Dashboards & Error Budgets	
In one line	Turns raw telemetry into the official availability score and shows how much "failure allowance" is left.
Simple analogy	A school report card that shows your attendance percentage and how many more absences are allowed.
What it does, step by step	• **SLI** (Service Level Indicator): what is measured, e.g., successful requests ÷ valid requests. • **SLO** (Service Level Objective): the target, here **99.99%** over a rolling 30 days. • **Error budget** = 100% − 99.99% = 0.01% of requests (about 4.3 minutes of full downtime per month). • Dashboards show availability per API, per cell and per tenant, plus budget remaining. • Counts 5xx errors and too-slow responses as failures; client mistakes (4xx) are not counted against the platform.
How it protects 99.99%	• This is the "as measured by SLO dashboards" part of the achievement: the number is objective and auditable. • When the budget runs low, teams freeze risky releases and focus on reliability work.
Built with	Grafana / OCI Monitoring dashboards, recording rules for SLIs.

#22 Burn-rate Alerting	
In one line	Wakes up an engineer only when the error budget is being used up dangerously fast.
Simple analogy	A smoke alarm that ignores burnt toast but rings for a real fire.
What it does, step by step	• **Burn rate** = how fast the budget is being spent compared to normal. Burn rate 1 = budget lasts exactly 30 days. • **Fast-burn alert** (e.g., burn rate ≥ 14.4 over 1 hour, confirmed over 5 minutes): pages on-call immediately. • **Slow-burn alert** (e.g., burn rate ≥ 6 over 6 hours): opens a ticket for the team. • Alerts are based on tenant-facing symptoms (errors, latency), not on every CPU spike.
How it protects 99.99%	• Fewer false alarms means engineers trust and react to real ones quickly. • Detects problems in minutes, which is essential when only ~4 minutes of downtime are allowed per month.

Built with	Prometheus Alertmanager / OCI Alarms, PagerDuty-style paging.

#23 On-call & Auto-remediation	
In one line	The people and automation that fix problems quickly, then learn from them.
Simple analogy	A fire brigade with a clear checklist for every type of emergency.
What it does, step by step	• **Auto-remediation** handles common issues without humans: restart pods, drain a bad node, roll back a release, shift traffic away from an AD. • **Runbooks**: step-by-step guides for each alert so any on-call engineer can respond quickly. • Incident process: detect → mitigate first (stop the bleeding) → find root cause → fix. • **Blameless post-incident reviews** create action items that prevent the same failure again.
How it protects 99.99%	• Lowers **MTTR** (mean time to recover); recovery speed matters more than never failing. • Regular **game days / chaos tests** (deliberately killing nodes or an AD) prove that failover really works.
Built with	Runbook automation, Kubernetes operators, incident management tooling.

4.8 Why Java for some services and Go for others

	Java	Go
Used for	API Gateway (#5), Write Service (#9)	Rate Limiter (#6), Router (#7), Read Service (#8), Workers (#10)
Strengths	Rich ecosystem, mature transaction and database libraries, strong typing for complex business rules	Tiny memory footprint, fast startup, very cheap concurrency (goroutines), simple deployment as one binary
Why it fits	Correctness-heavy logic: validation, transactions, outbox, idempotency	High-throughput, latency-sensitive hops and many concurrent jobs
Reliability tuning	Low-pause garbage collector (e.g., G1/ZGC), bounded thread pools, connection pools	Context deadlines, bounded worker pools, quick scale-out thanks to fast startup

5. End-to-End Workflows

Eight workflows cover everything the platform does: serving requests, background work, onboarding, scaling, surviving failures, releasing code, and measuring availability. The "Who" column uses the component numbers from the diagram.

5.1 Write request (saving data), the main path

This is the most important journey, because it must be both fast **and** durable. The sequence diagram on the next page shows the same steps visually.

Step	Who	What happens (in simple terms)
1	Tenant app (#1)	The app sends **PUT/POST https://api…/v1/items** with an access token and an idempotency key (a unique ID for this action).
2	DNS (#2)	The app's computer asks DNS for the API's address. DNS returns the address of a healthy load balancer in the nearest healthy region.
3	WAF (#3)	The request is screened. Attacks, malformed traffic and abusive IPs are blocked here.
4	Load Balancer (#4)	HTTPS is decrypted (TLS termination). The request is sent to a healthy API Gateway pod in one of the 3 ADs.
5	API Gateway (#5)	Verifies the token signature using cached keys, reads the **tenant ID**, validates the request, adds a **trace ID** and a deadline (e.g., 2 seconds total).
6	Rate Limiter (#6)	Takes one token from the tenant's bucket. If empty → reply **429 + Retry-After**. Otherwise → allowed.
7	Router (#7)	Looks up the tenant's **cell** in its locally cached routing map and calls a healthy Write Service pod in that cell (circuit breaker and retry rules apply).
8	Write Service (#9)	Checks the idempotency key. Already seen? → return the saved earlier result (no duplicate). New? → validate business rules.
9	Database (#12)	In one transaction, saves the data **and** an outbox event row. The commit succeeds only when **2 of 3 replicas** (in different ADs) confirm.
10	Write Service (#9)	Commit confirmed → the data is now durable. Stores the response against the idempotency key.
11	Outbox relay → Queue (#13)	The outbox relay publishes the event to the tenant's partition in the message queue (replicated, retained for days).
12	Back to the tenant	Response **201 Created** travels back through Router → Gateway → Load Balancer → tenant app. Typical total time: tens of milliseconds.

Step	Who	What happens (in simple terms)
13	Every service	Each hop records metrics (count, latency, status), a log line and a trace span, all tagged with tenant ID and trace ID → Telemetry (#20).

5.2 Workflow diagram: a write request from start to finish

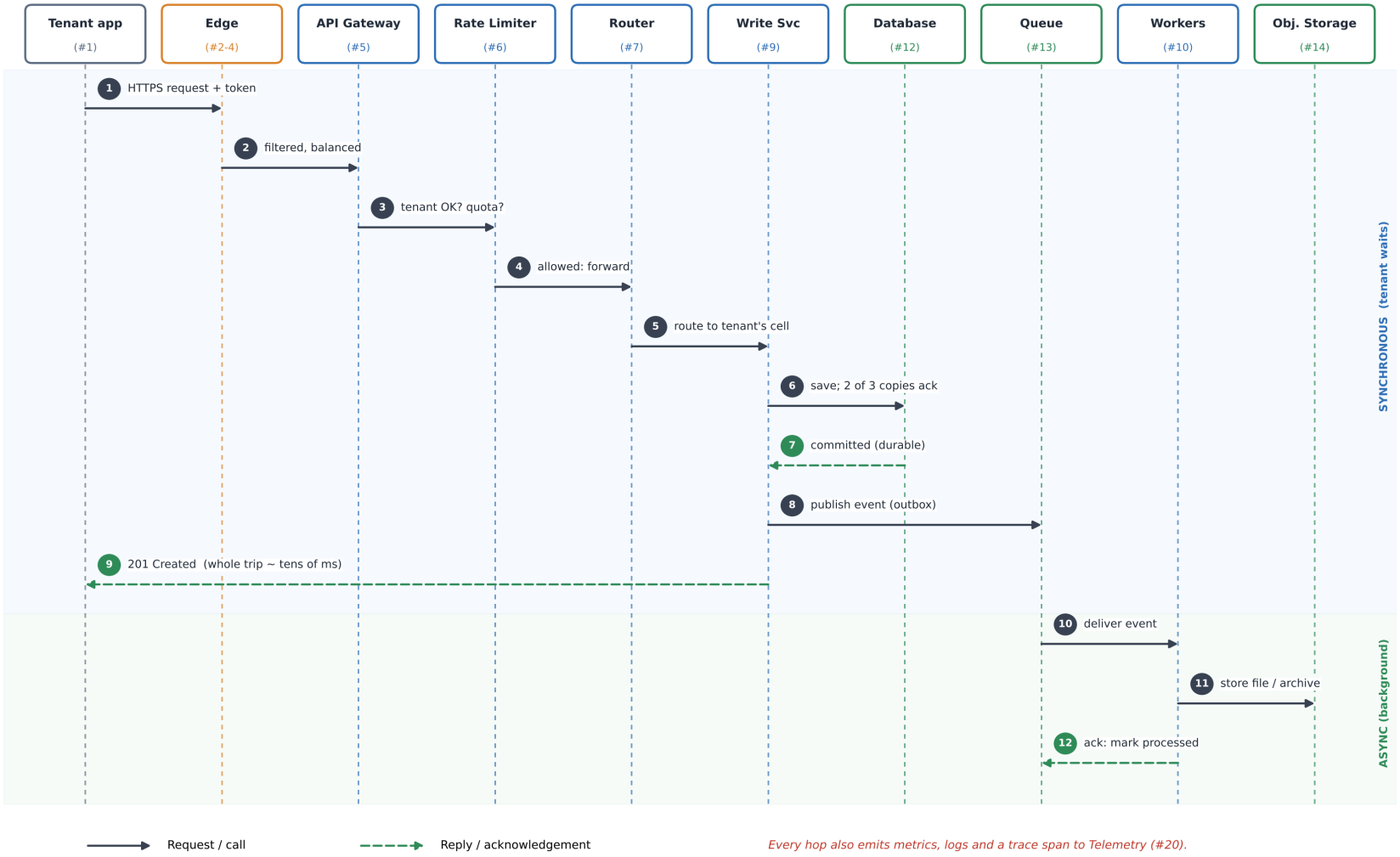


Figure 2. Sequence of a write request. Steps 1–9 happen while the tenant waits; steps 10–12 run in the background.

How to read Figure 2	Explanation
Boxes on top	Each box is a participant (with its component number). The dashed vertical line under it is its "lifeline" (time flows downwards).
Solid arrows (1–6, 8, 10, 11)	A participant calling the next one. 1: app sends request; 2: edge passes clean traffic; 3–4: gateway checks tenant and quota; 5: router picks the cell; 6: write saved to 3 database copies; 8: event published; 10–11: worker processes it.
Green dashed arrows (7, 9, 12)	Replies. 7: database confirms the write is durable; 9: tenant receives 201 Created; 12: worker acknowledges the event so it is not delivered again.
Blue band vs green band	Blue = synchronous (the tenant is waiting). Green = asynchronous (background; the tenant already has the answer).

5.3 Background processing (async workflow)

Step	Who	What happens (in simple terms)
1	Queue (#13)	Delivers the event to one Async Worker (#10) in the tenant's consumer group.
2	Worker (#10)	Checks whether this event was already processed (idempotency). If yes, it just acknowledges it.
3	Worker (#10)	Does the work: e.g., builds search index, updates aggregates, writes a file/export to Object Storage (#14), sends a notification.
4	Worker (#10)	Success → **acknowledges** (commits the offset). The event will not be delivered again.
5	On failure	Retries with exponential backoff (e.g., 1s, 2s, 4s…). After N attempts → event moved to the **dead-letter queue** and an alert/ticket is raised. Other events keep flowing.
6	If the worker crashes	The event was never acknowledged, so the queue gives it to another worker. Because the job is idempotent, re-running it is safe.
7	Autoscaler (#18)	If **queue lag** grows (events waiting), more worker pods are added; when the lag drops, they are removed.

5.4 Read request (fetching data)

Step	Who	What happens (in simple terms)
1–6	Same as write path	DNS → WAF → Load Balancer → API Gateway → Rate Limiter → Router (identity, tenant, quota checks).
7	Router (#7)	Sends the request to a Read Service (#8) pod in the tenant's cell. If it is slow, a **hedged** second request may go to another pod.
8	Read Service (#8)	Looks in the Cache (#11) using a tenant-prefixed key. **Hit** → return immediately (sub-millisecond lookup).
9	On cache miss	Reads from a database replica (#12), always filtered by tenant ID; stores the result in the cache with a TTL.
10	If cache is down	Skips the cache and reads from the database directly: slower but still correct (graceful degradation).
11	Back to the tenant	**200 OK** with the data. Metrics and traces are recorded as before.

5.5 Tenant onboarding (control-plane workflow)

Step	Who	What happens (in simple terms)
1	Admin / signup	A new tenant is created through the control-plane API.
2	Config Service (#16)	Creates a tenant ID, picks the plan's quotas, and assigns the tenant to a **cell** with spare capacity.
3	Identity (#17)	Creates the tenant's identity, credentials and (optionally) a dedicated encryption key in Vault.
4	Data layer	Creates the tenant's partition/schema in the database and buckets in Object Storage.
5	Config push	The new routing entry and quotas are pushed to Routers and Rate Limiters, which cache them locally.
6	Ready	The tenant can now send requests. Existing tenants were never affected, because the control plane is separate from the request path.

5.6 Elasticity: handling a traffic spike

Step	Who	What happens (in simple terms)
1	Traffic rises	A big tenant starts a campaign; requests per second double in a few minutes.
2	Rate Limiter (#6)	Keeps that tenant within its quota so others are not slowed down.
3	Warm headroom	Spare capacity already running absorbs the first part of the spike immediately.
4	Telemetry (#20)	CPU, requests per second and latency metrics rise.
5	HPA (#18)	Adds more pods for the busy services within seconds to a minute.
6	Cluster Autoscaler (#18)	If new pods cannot fit, adds new nodes (minutes). Pods are spread across ADs and FDs.
7	Load Balancer / Router	Automatically start sending traffic to the new pods once their health checks pass.
8	Traffic falls	After a cool-down period, extra pods and nodes are removed to save cost (never below the N+1 minimum).

5.7 Failure and self-healing

Different failures, from small to large, and what the platform does automatically.

Case	What fails	What happens (in simple terms)
A	A pod crashes	Kubernetes restarts it in seconds. Health checks remove it from load balancing meanwhile; other copies serve traffic. **Tenant impact: none.**
B	A node dies	All its pods are rescheduled onto other nodes. Because copies were spread across nodes/FDs, service continues. **Tenant impact: a few retried requests at most.**
C	A database primary fails	A replica in another AD is promoted automatically within seconds. Acknowledged writes are safe because they were on a quorum. Writes may briefly retry.
D	A full AD goes down	Load balancer and router stop sending traffic there. The other 2 ADs already have ~66%-max utilisation headroom, so they carry 100% of traffic; autoscaler adds more if needed.
E	A bad release	Canary metrics worsen → automatic rollback within minutes; only a small slice of traffic was ever exposed.
F	Control plane outage	Data plane keeps serving using the last-known-good config (static stability). Only onboarding/config changes pause.
G	Whole region disaster (rare)	DNS steering (#2) moves traffic to a standby region; data restored from cross-region replicas/backups (#12, #14).

5.8 Safe deployment of new code

Step	Who	What happens (in simple terms)
1	Developer	Merges code after review; pipeline (#19) builds and runs automated tests.
2	Pre-production	Deployed to staging; integration and load tests run against production-like traffic.
3	Canary	Released to one cell / a small % of production traffic.
4	Bake & compare	For a set time, canary error rate and latency are compared with the old version.
5	Decision	Healthy → continue. Worse → **automatic rollback** and the team is notified.
6	Waves	Rolled out one AD at a time, then region by region, with bake time between waves.
7	Budget gate	If the error budget is nearly used up, non-urgent releases are frozen.

5.9 How the 99.99% is measured

Step	Who	What happens (in simple terms)
1	Count	Every request is counted as **good** (success within the latency target) or **bad** (5xx error or too slow).
2	Probe	Synthetic probes call the API every minute from outside, catching problems even when real traffic is low.
3	Compute SLI	Availability = good requests ÷ valid requests, over a rolling 30-day window.
4	Compare to SLO	Target = 99.99%. Error budget = 0.01% of requests.
5	Burn rate	Measures how fast the budget is being used; fast burn → page, slow burn → ticket.
6	Review	Weekly reliability review of dashboards; post-incident actions are tracked to completion.
7	Result	The dashboards showed **≥ 99.99%** sustained availability across the 12K+-node platform.

6. The Requirements I Defined

A big part of the role was writing clear, testable requirements that every service team had to meet. Example targets are shown; replace them with your real numbers.

Requirement	Plain meaning	Example target	How the design meets it
Availability	The service works when tenants need it.	99.99% of valid requests succeed, rolling 30 days.	Redundancy across 3 ADs, cells, safe deploys, static stability, burn-rate alerts.
Scalability	Handles more tenants and traffic by adding machines.	Linear scale to 2–3× peak; no single bottleneck; shards for data.	Stateless services, sharded database, partitioned queue, cell-based growth (add new cells).
Elasticity	Grows and shrinks automatically, quickly.	Absorb 2× spike in under 5 min; scale down when idle.	HPA + cluster autoscaler, warm headroom, predictive scaling, queue-lag scaling.
Durability	Saved data is never lost.	Zero acknowledged-write loss; RPO ≈ 0 within region; tested restores.	Quorum writes, outbox, replicated queue, backups + PITR, cross-region copies.
Latency	Requests are fast.	e.g., p99 < 200 ms for reads, < 500 ms for writes.	Cache, hedged reads, async heavy work, deadlines and timeouts.
Isolation	One tenant can't hurt another.	No tenant > its quota; per-tenant data separation.	Rate limiter, cells, tenant ID on every query and key.
Recoverability	Quick recovery from failure.	Automated recovery for common faults; MTTR in minutes.	Auto-remediation, runbooks, rollbacks, regular game days.

7. The 99.99% in Numbers

7.1 How much downtime each target allows

Target	Per year	Per 30-day month	Per day
99.9% (three nines)	8 h 46 min	43 min 12 s	1 min 26 s
99.99% (four nines)	52 min 34 s	4 min 19 s	8.6 s
99.999% (five nines)	5 min 15 s	26 s	0.9 s

7.2 A worked example

- Suppose the platform handles **1 billion requests per month**.
- Error budget = 0.01% × 1,000,000,000 = **100,000 failed requests allowed** per month.
- If an incident causes 2% of requests to fail for 10 minutes at 400 requests/second: 400 × 600 s × 2% = **4,800 failures**, about 5% of the monthly budget.
- That is why fast detection (burn-rate alerts) and automatic rollback matter so much.

7.3 Why three Availability Domains and ~66% utilisation

With 3 ADs sharing traffic equally, each carries one third. If one AD fails, the other two must carry everything, so each must be able to handle half of the total. That means in normal times each AD should run at no more than about **2/3 (≈66%)** of its capacity. This "N+1" rule is what turns an AD outage into a non-event for tenants.

7.4 Why redundancy multiplies availability

If one copy of a component is available 99% of the time, two independent copies are both down only 1% × 1% = 0.01% of the time, giving 99.99%. Three copies push this even further. Spreading copies across independent ADs and FDs is what keeps those failures independent.

8. How to Explain This in an Interview

8.1 The 30-second version

> **Say it like this**
>
> "I led the design of the request-handling services for a multi-tenant platform on OCI running on more than 12,000 nodes. I designed Java and Go microservices (a gateway, rate limiter, cell router, read/write services and async workers) and defined the scalability, elasticity and durability requirements every team had to meet. Key ideas were cell-based isolation, quorum writes with idempotency, separating the control plane from the data plane, N+1 capacity across three Availability Domains, and canary deployments with automatic rollback. We measured success with SLO dashboards and burn-rate alerting, and sustained 99.99% availability: about four minutes of allowed downtime a month."

8.2 STAR summary

Part	Details
Situation	A shared, multi-tenant platform on OCI at 12K+ nodes, where one tenant or one failure could affect many customers.
Task	Lead the data-plane design and define the scalability, elasticity and durability requirements needed to hit 99.99%.
Action	Designed Java/Go microservices; introduced cells, rate limiting, quorum writes, outbox + idempotency, static stability, N+1 AD capacity, autoscaling, safe deployment and SLO-based alerting.
Result	99.99% availability sustained, as measured by SLO dashboards; failures stayed contained; scaling became automatic.

8.3 Likely follow-up questions

- **How did you stop noisy neighbours?** Per-tenant token-bucket rate limits, quotas from the config service, and cell isolation.
- **What happens if an AD goes down?** Traffic shifts to the other two ADs, which already have headroom; database replicas are promoted automatically.
- **How do you avoid duplicate writes on retry?** Idempotency keys checked by the Write Service, plus idempotent workers.
- **Why not make everything synchronous?** It would be slower and fragile; the queue absorbs spikes and isolates slow downstream work.
- **How do you know it's really 99.99%?** Request-based SLIs from server metrics plus external synthetic probes, over a rolling 30-day window.